Pack 19 (Desktop) — File-Transfer Resumption Guards + Clipboard Resync After Reconnect (real code)

RemoteDesk · Pack 19 (Desktop) — File-Transfer Resumption Guards + Clipboard Resync After Reconnect (real code)

> Codex / Claude Code handoff. Direct continuation of Desktop Packs 15 → 18. After Pack 18 made reconnects re-check policy before resuming control, this slice handles the two stateful interactive features across a reconnect: in-flight FILE TRANSFERS must pause on drop and only resume after a policy re-check (and only if file transfer is still permitted), and CLIPBOARD sync must invalidate cleanly so a stale clipboard can't replay post-reconnect. Build order: shared transfer/clipboard contracts → transfer resume guard → clipboard resync guard → reconnect wiring → UI → tests.
> Scope: make file transfer + clipboard reconnect-safe. **Fail-closed everywhere. Transfers never auto-resume on a revoked/blocked policy. No stale clipboard replay. Host stays source of truth. No native executor. No secrets/payloads in logs.**
> Tag legend: **[NEW]** = create file. **[MODIFY]** = patch existing file by hand (merge, don't overwrite).

Files created / modified / not touched

```
CREATED [NEW]
  packages/shared/src/permissions/fileTransferState.ts
  packages/shared/src/permissions/clipboardSync.ts
  apps/desktop/src/renderer/src/services/fileTransferResumeGuard.ts
  apps/desktop/src/renderer/src/services/clipboardResyncGuard.ts
  apps/desktop/src/renderer/src/components/TransferResumePanel.tsx
  tests/permissions/fileTransferState.test.ts
  tests/permissions/fileTransferResumeGuard.test.ts
  tests/permissions/clipboardSync.test.ts

MODIFIED [MODIFY]
  packages/shared/src/permissions/index.ts                          (export fileTransferState + clipboardSync)
  apps/desktop/src/renderer/src/services/reconnectPolicyController.ts (notify transfer + clipboard guards)
  apps/desktop/src/renderer/src/services/sessionDataChannel.ts       (route transfer/clipboard through guards)
  apps/desktop/src/renderer/src/components/RemoteSessionView.tsx      (transfer resume panel)

NOT TOUCHED (intentionally)
  apps/desktop/src/main/index.ts          (no native executor)
  apps/desktop/src/preload/index.ts       (no new bridge)
  apps/desktop/src/renderer/src/services/webrtc.ts          (Pack 15/17 input gates unchanged)
  apps/desktop/src/renderer/src/services/permissionState.ts (Pack 15/16 unchanged)
  apps/api/**  apps/web/**
```

Part A — Shared: file-transfer + clipboard contracts

FILE: packages/shared/src/permissions/fileTransferState.ts

```
import { canEnableFileTransfer, type DeviceAccessPolicySnapshot } from './policyEvaluator';

/**
 * Reconnect-safe state model for an in-flight file transfer. On a transport drop
 * an active transfer becomes `paused`; it may only return to `active` after a
 * policy re-check that still permits file transfer. Fail closed: missing policy
 * or a denial moves it to `denied`, never back to active.
 */
export type FileTransferStatus =
  | 'idle'
  | 'active'
  | 'paused'       // transport dropped mid-transfer
  | 'resuming'     // policy re-check in progress
  | 'denied'       // policy no longer permits -> cannot resume
  | 'completed'
  | 'cancelled';

export interface FileTransferSnapshot {
  transferId: string;
  status: FileTransferStatus;
  /** Bytes confirmed transferred (for resume offset + UI). */
  bytesTransferred: number;
  totalBytes: number;
  /** Non-sensitive display name only. */
  fileName: string;
  updatedAt: string;
}

export function transferProgress(s: FileTransferSnapshot): number {
  if (s.totalBytes <= 0) return 0;
  return Math.min(1, s.bytesTransferred / s.totalBytes);
}

/** A transfer is in-flight (and thus pausable) only when active or resuming. */
export function isInFlight(status: FileTransferStatus): boolean {
  return status === 'active' || status === 'resuming';
}

/**
 * Decide the next status when the transport drops. Only in-flight transfers
 * pause; everything else is unaffected.
 */
export function onTransportDrop(status: FileTransferStatus): FileTransferStatus {
  return isInFlight(status) ? 'paused' : status;
}

/**
 * Decide whether a paused transfer may resume after a reconnect, given the fresh
 * policy + host acceptance. Fail closed: only a paused transfer with an allowing
 * policy resumes; otherwise it is denied.
 */
export function canResumeTransfer(
  status: FileTransferStatus,
```

```
  policy: DeviceAccessPolicySnapshot | null,
  hostAccepted: boolean,
): boolean {
  if (status !== 'paused') return false;
  return canEnableFileTransfer(policy, hostAccepted).allowed;
}
```

FILE: packages/shared/src/permissions/clipboardSync.ts

```
import { canEnableClipboard, type DeviceAccessPolicySnapshot } from './policyEvaluator';

/**
 * Clipboard reconnect safety. Each clipboard payload carries an epoch + sequence.
 * On reconnect the epoch is bumped, so any in-flight or replayed frame from the
 * previous connection is rejected (stale). This prevents a clipboard captured
 * before a policy change from landing after it.
 */
export interface ClipboardEnvelopeMeta {
  epoch: number;
  seq: number;
}

/** Bump on every (re)connect so older epochs are considered stale. */
export function nextEpoch(current: number): number {
  return current + 1;
}

/** Is an incoming clipboard frame fresh for the current epoch? */
export function isClipboardFresh(
  incoming: ClipboardEnvelopeMeta,
  currentEpoch: number,
  lastSeqForEpoch: number,
): boolean {
  if (incoming.epoch !== currentEpoch) return false; // stale from a prior connection
  return incoming.seq > lastSeqForEpoch;             // strictly newer within epoch
}

/**
 * Whether clipboard sync may operate at all right now. Fail closed via the shared
 * evaluator: missing policy / blocked / disabled / not-accepted => false.
 */
export function clipboardOperational(
  policy: DeviceAccessPolicySnapshot | null,
  hostAccepted: boolean,
): boolean {
  return canEnableClipboard(policy, hostAccepted).allowed;
}
```

FILE: packages/shared/src/permissions/index.ts [MODIFY]

```
export * from './devicePolicy';
export * from './policyEvaluator';
export * from './sessionApproval';
export * from './inputAction';
export * from './sessionAuditEvents';
export * from './reconnect';
```

```
export * from './fileTransferState';  // ADD
export * from './clipboardSync';       // ADD
```

Part B — Desktop: file-transfer resume guard

FILE: apps/desktop/src/renderer/src/services/fileTransferResumeGuard.ts

```
import {
  onTransportDrop,
  canResumeTransfer,
  makeAuditEvent,
  type DeviceAccessPolicySnapshot,
  type FileTransferSnapshot,
  type SessionAuditEvent,
} from '@remotedesk/shared';

/**
 * Tracks in-flight file transfers across a reconnect. On drop, in-flight
 * transfers are paused. On reconnect the controller calls resumeAll() with the
 * freshly re-checked policy + host acceptance; only transfers the policy still
 * permits resume, the rest are denied. Fail closed.
 */
export interface TransferResumeDeps {
  /** Resume the underlying transfer from its byte offset (existing transfer impl). */
  resumeTransfer: (transferId: string, fromByte: number) => void;
  /** Cancel/abort the underlying transfer. */
  abortTransfer: (transferId: string) => void;
  onAudit?: (e: SessionAuditEvent) => void;
}

export class FileTransferResumeGuard {
  private transfers = new Map<string, FileTransferSnapshot>();
  private listeners = new Set<() => void>();

  constructor(private readonly deps: TransferResumeDeps) {}

  /** Register/refresh a transfer snapshot (called as the transfer progresses). */
  track(snapshot: FileTransferSnapshot): void {
    this.transfers.set(snapshot.transferId, snapshot);
    this.emit();
  }

  remove(transferId: string): void {
    this.transfers.delete(transferId);
    this.emit();
  }

  list(): FileTransferSnapshot[] {
    return [...this.transfers.values()];
  }

  /** Transport dropped: pause all in-flight transfers. */
  pauseInFlight(): void {
    for (const [id, snap] of this.transfers) {
      const status = onTransportDrop(snap.status);
```

```
      if (status !== snap.status) {
        this.transfers.set(id, { ...snap, status, updatedAt: new Date().toISOString() });
      }
    }
    this.emit();
  }

  /**
   * Reconnect re-check done: resume permitted paused transfers, deny the rest.
   * Returns counts for the UI/audit.
   */
  resumeAll(
    policy: DeviceAccessPolicySnapshot | null,
    hostAccepted: boolean,
  ): { resumed: number; denied: number } {
    let resumed = 0;
    let denied = 0;
    for (const [id, snap] of this.transfers) {
      if (snap.status !== 'paused') continue;
      if (canResumeTransfer(snap.status, policy, hostAccepted)) {
        this.transfers.set(id, { ...snap, status: 'resuming', updatedAt: new Date().toISOString() });
        this.deps.resumeTransfer(id, snap.bytesTransferred);
        resumed += 1;
      } else {
        this.transfers.set(id, { ...snap, status: 'denied', updatedAt: new Date().toISOString() });
        this.deps.abortTransfer(id);
        denied += 1;
      }
    }
    if (resumed || denied) {
      this.deps.onAudit?.(makeAuditEvent('control.revoked', { scope: 'file_transfer', resumed, denied }));
      this.emit();
    }
    return { resumed, denied };
  }

  subscribe(fn: () => void): () => void {
    this.listeners.add(fn);
    return () => this.listeners.delete(fn);
  }

  private emit(): void {
    for (const fn of this.listeners) fn();
  }
}
```

Part C — Desktop: clipboard resync guard

FILE: apps/desktop/src/renderer/src/services/clipboardResyncGuard.ts

```
import {
  nextEpoch,
  isClipboardFresh,
  clipboardOperational,
  type ClipboardEnvelopeMeta,
```

```
  type DeviceAccessPolicySnapshot,
} from '@remotedesk/shared';

/**
 * Guards clipboard frames across reconnects. Holds the current epoch + the last
 * accepted sequence per epoch. On reconnect, bumpEpoch() invalidates every frame
 * from the prior connection, so a clipboard captured before a policy change can
 * never replay afterward. Fail closed: when clipboard is not operational, all
 * frames are rejected.
 */
export class ClipboardResyncGuard {
  private epoch = 1;
  private lastSeqForEpoch = 0;
  private operational = false;

  /** Update whether clipboard sync is currently permitted by policy. */
  refreshOperational(policy: DeviceAccessPolicySnapshot | null, hostAccepted: boolean): void {
    this.operational = clipboardOperational(policy, hostAccepted);
  }

  /** Call on every (re)connect: new epoch, reset per-epoch sequence. */
  bumpEpoch(): void {
    this.epoch = nextEpoch(this.epoch);
    this.lastSeqForEpoch = 0;
  }

  currentEpoch(): number {
    return this.epoch;
  }

  /** Stamp an outgoing clipboard frame with the current epoch + next seq. */
  stampOutgoing(): ClipboardEnvelopeMeta | null {
    if (!this.operational) return null; // fail closed: don't send when not allowed
    this.lastSeqForEpoch += 1;
    return { epoch: this.epoch, seq: this.lastSeqForEpoch };
  }

  /** Decide whether an incoming clipboard frame should be applied. */
  acceptIncoming(meta: ClipboardEnvelopeMeta): boolean {
    if (!this.operational) return false; // fail closed
    if (!isClipboardFresh(meta, this.epoch, this.lastSeqForEpoch)) return false;
    this.lastSeqForEpoch = meta.seq;
    return true;
  }
}
```

Part D — Desktop: reconnect + data-channel wiring

FILE: apps/desktop/src/renderer/src/services/reconnectPolicyController.ts [MODIFY]

```
// ...existing Pack 18 imports...
import type { FileTransferResumeGuard } from './fileTransferResumeGuard';
import type { ClipboardResyncGuard } from './clipboardResyncGuard';

// ---- MODIFY: accept the two guards in ReconnectDeps (all optional) ----
```

```
// export interface ReconnectDeps {
//   ...existing...
//   fileTransferGuard?: FileTransferResumeGuard;  // ADD
//   clipboardGuard?: ClipboardResyncGuard;          // ADD
//   /** Current host acceptance, for resume decisions. */
//   hostAccepted?: () => boolean;                    // ADD
// }

// ---- MODIFY: in onDrop(), pause transfers + invalidate clipboard immediately ----
// onDrop(): void {
//   this.deps.revokeInteractive();
//   this.deps.fileTransferGuard?.pauseInFlight();   // pause in-flight transfers
//   this.deps.clipboardGuard?.bumpEpoch();           // stale-out old clipboard frames
//   this.deps.onAudit?.(makeAuditEvent('control.revoked', { reason: 'transport_drop' }));
//   this.transition('reconnecting', { attempt: this.state.attempt + 1 });
// }

// ---- MODIFY: in onTransportRestored(), after policy re-check passes, resume ----
// async onTransportRestored(): Promise<boolean> {
//   this.transition('rechecking_policy');
//   const policy = await this.deps.refetchPolicy();
//   this.deps.applyPolicy(policy);
//   const decision = evaluateReconnectResume(policy);
//   if (!decision.allowed) { this.transition('failed', { failureReason: decision.reason }); return false; }
//
//   const accepted = this.deps.hostAccepted?.() ?? false;
//   this.deps.clipboardGuard?.bumpEpoch();                       // fresh epoch for the new connection
//   this.deps.clipboardGuard?.refreshOperational(policy, accepted);
//   this.deps.fileTransferGuard?.resumeAll(policy, accepted);    // resume permitted, deny the rest
//   this.transition('recovered');
//   return true;
// }
```

FILE: apps/desktop/src/renderer/src/services/sessionDataChannel.ts [MODIFY]

```
// ...existing imports + Pack 16 channelFeatureGate...
import type { ClipboardEnvelopeMeta } from '@remotedesk/shared';
import type { ClipboardResyncGuard } from './clipboardResyncGuard';
import type { FileTransferResumeGuard } from './fileTransferResumeGuard';

// ---- ADD: clipboard frames now carry envelope meta (epoch + seq) ----
// Extend your ClipboardFrame payload with: meta: ClipboardEnvelopeMeta

// ---- MODIFY: clipboard send path — stamp + gate ----
// export function sendClipboard(data: ClipboardPayload, clipboardGuard: ClipboardResyncGuard): void {
//   if (!channelFeatureGate.guardClipboard()) return;       // Pack 16 policy gate
//   const meta = clipboardGuard.stampOutgoing();
//   if (!meta) return;                                       // not operational -> drop
//   clipboardChannel.send(serializeClipboard({ ...data, meta }));
// }

// ---- MODIFY: clipboard receive path — freshness + gate ----
// clipboardChannel.on('clipboard', (frame, clipboardGuard: ClipboardResyncGuard) => {
//   if (!channelFeatureGate.guardClipboard()) return;
//   if (!clipboardGuard.acceptIncoming(frame.meta as ClipboardEnvelopeMeta)) return; // stale/denied
//   applyClipboard(frame);
```

```
// });

// ---- MODIFY: file-transfer progress now feeds the resume guard ----
// fileTransfer.onProgress((t) => {
//   fileTransferGuard.track({
//     transferId: t.id, status: 'active', bytesTransferred: t.bytes,
//     totalBytes: t.total, fileName: t.name, updatedAt: new Date().toISOString(),
//   });
// });
// fileTransfer.onComplete((t) => fileTransferGuard.remove(t.id));
//
// The existing guardFileTransfer() request gate from Pack 16 stays as-is.
```

Part E — UI

FILE: apps/desktop/src/renderer/src/components/TransferResumePanel.tsx

```
import React, { useEffect, useState } from 'react';
import { transferProgress, type FileTransferSnapshot } from '@remotedesk/shared';
import type { FileTransferResumeGuard } from '../services/fileTransferResumeGuard';

const STATUS_LABEL: Record<string, string> = {
  active: 'Transferring', paused: 'Paused (reconnecting)', resuming: 'Resuming',
  denied: 'Blocked by policy', completed: 'Done', cancelled: 'Cancelled', idle: 'Idle',
};

/**
 * Read-only list of file transfers + their reconnect state. Shows when a transfer
 * paused on a drop and whether it resumed or was denied after the policy re-check.
 */
export function TransferResumePanel({ guard }: { guard: FileTransferResumeGuard }) {
  const [transfers, setTransfers] = useState<FileTransferSnapshot[]>(guard.list());

  useEffect(() => guard.subscribe(() => setTransfers(guard.list())), [guard]);

  if (transfers.length === 0) return null;
  return (
    <ul className="transfer-resume-panel">
      {transfers.map((t) => (
        <li key={t.transferId} className={`transfer-item transfer-item--${t.status}`}>
          <span className="transfer-item__name">{t.fileName}</span>
          <span className="transfer-item__status">{STATUS_LABEL[t.status] ?? t.status}</span>
          <span className="transfer-item__pct">{Math.round(transferProgress(t) * 100)}%</span>
        </li>
      ))}
    </ul>
  );
}
```

FILE: apps/desktop/src/renderer/src/components/RemoteSessionView.tsx [MODIFY]

```
// ...existing imports + Pack 15/16/17/18 additions...
import { TransferResumePanel } from './TransferResumePanel';
// fileTransferResumeGuard + clipboardResyncGuard are constructed at session start
// and passed into the ReconnectPolicyController deps (Part D wiring).
```

```
// ---- ADD to render: transfer resume panel near the timeline ----
// <TransferResumePanel guard={fileTransferResumeGuard} />
//
// Existing ReconnectBanner / SessionTimelinePanel / PolicyStatusBanner /
// EmergencyStopButton / SessionApprovalPrompt / SessionExpiryBadge /
// InputActivityMeter all stay as-is.
```

Part F — Tests

FILE: tests/permissions/fileTransferState.test.ts

```
import { describe, it, expect } from 'vitest';
import {
  transferProgress,
  isInFlight,
  onTransportDrop,
  canResumeTransfer,
  type DeviceAccessPolicySnapshot,
  type FileTransferSnapshot,
} from '@remotedesk/shared';

function policy(over: Partial<DeviceAccessPolicySnapshot> = {}): DeviceAccessPolicySnapshot {
  return {
    deviceId: 'dev-1', trustStatus: 'trusted', unattendedAccessEnabled: false,
    remoteInputEnabled: true, clipboardSyncEnabled: true, fileTransferEnabled: true,
    requiresSessionApproval: true, maxSessionMinutes: null, updatedAt: new Date().toISOString(),
    ...over,
  };
}

describe('fileTransferState', () => {
  it('computes progress', () => {
    const s: FileTransferSnapshot = { transferId: 't', status: 'active', bytesTransferred: 50, totalBytes: 200, fileName:
'a.bin', updatedAt: '' };
    expect(transferProgress(s)).toBe(0.25);
  });

  it('only active/resuming are in-flight', () => {
    expect(isInFlight('active')).toBe(true);
    expect(isInFlight('resuming')).toBe(true);
    expect(isInFlight('paused')).toBe(false);
  });

  it('pauses only in-flight transfers on drop', () => {
    expect(onTransportDrop('active')).toBe('paused');
    expect(onTransportDrop('completed')).toBe('completed');
    expect(onTransportDrop('idle')).toBe('idle');
  });

  it('resumes a paused transfer only when policy permits + host accepted', () => {
    expect(canResumeTransfer('paused', policy(), true)).toBe(true);
    expect(canResumeTransfer('paused', policy(), false)).toBe(false);        // host not accepted
    expect(canResumeTransfer('paused', policy({ fileTransferEnabled: false }), true)).toBe(false);
    expect(canResumeTransfer('paused', null, true)).toBe(false);             // missing policy
```

```
    expect(canResumeTransfer('paused', policy({ trustStatus: 'blocked' }), true)).toBe(false);
  });

  it('never resumes a non-paused transfer', () => {
    expect(canResumeTransfer('active', policy(), true)).toBe(false);
  });
});
```

FILE: tests/permissions/fileTransferResumeGuard.test.ts

```
import { describe, it, expect, vi } from 'vitest';
import { FileTransferResumeGuard } from '../../apps/desktop/src/renderer/src/services/fileTransferResumeGuard';
import type { DeviceAccessPolicySnapshot, FileTransferSnapshot } from '@remotedesk/shared';

function policy(over: Partial<DeviceAccessPolicySnapshot> = {}): DeviceAccessPolicySnapshot {
  return {
    deviceId: 'dev-1', trustStatus: 'trusted', unattendedAccessEnabled: false,
    remoteInputEnabled: true, clipboardSyncEnabled: true, fileTransferEnabled: true,
    requiresSessionApproval: true, maxSessionMinutes: null, updatedAt: new Date().toISOString(),
    ...over,
  };
}
function snap(id: string, status: FileTransferSnapshot['status'] = 'active'): FileTransferSnapshot {
  return { transferId: id, status, bytesTransferred: 100, totalBytes: 1000, fileName: `${id}.bin`, updatedAt: '' };
}

describe('FileTransferResumeGuard', () => {
  it('pauses in-flight transfers on drop', () => {
    const guard = new FileTransferResumeGuard({ resumeTransfer: vi.fn(), abortTransfer: vi.fn() });
    guard.track(snap('a', 'active'));
    guard.track(snap('b', 'completed'));
    guard.pauseInFlight();
    expect(guard.list().find((t) => t.transferId === 'a')!.status).toBe('paused');
    expect(guard.list().find((t) => t.transferId === 'b')!.status).toBe('completed');
  });

  it('resumes permitted transfers and denies the rest after re-check', () => {
    const resumeTransfer = vi.fn();
    const abortTransfer = vi.fn();
    const guard = new FileTransferResumeGuard({ resumeTransfer, abortTransfer });
    guard.track(snap('a', 'active'));
    guard.pauseInFlight();

    const res = guard.resumeAll(policy(), true);
    expect(res).toEqual({ resumed: 1, denied: 0 });
    expect(resumeTransfer).toHaveBeenCalledWith('a', 100);
    expect(guard.list()[0].status).toBe('resuming');
  });

  it('denies + aborts when policy no longer permits', () => {
    const resumeTransfer = vi.fn();
    const abortTransfer = vi.fn();
    const guard = new FileTransferResumeGuard({ resumeTransfer, abortTransfer });
    guard.track(snap('a', 'active'));
    guard.pauseInFlight();
```

```
    const res = guard.resumeAll(policy({ fileTransferEnabled: false }), true);
    expect(res).toEqual({ resumed: 0, denied: 1 });
    expect(abortTransfer).toHaveBeenCalledWith('a');
    expect(guard.list()[0].status).toBe('denied');
  });

  it('fails closed on a missing policy', () => {
    const guard = new FileTransferResumeGuard({ resumeTransfer: vi.fn(), abortTransfer: vi.fn() });
    guard.track(snap('a', 'active'));
    guard.pauseInFlight();
    expect(guard.resumeAll(null, true)).toEqual({ resumed: 0, denied: 1 });
  });
});
```

FILE: tests/permissions/clipboardSync.test.ts

```
import { describe, it, expect } from 'vitest';
import {
  nextEpoch,
  isClipboardFresh,
  clipboardOperational,
  type DeviceAccessPolicySnapshot,
} from '@remotedesk/shared';
import { ClipboardResyncGuard } from '../../apps/desktop/src/renderer/src/services/clipboardResyncGuard';

function policy(over: Partial<DeviceAccessPolicySnapshot> = {}): DeviceAccessPolicySnapshot {
  return {
    deviceId: 'dev-1', trustStatus: 'trusted', unattendedAccessEnabled: false,
    remoteInputEnabled: true, clipboardSyncEnabled: true, fileTransferEnabled: true,
    requiresSessionApproval: true, maxSessionMinutes: null, updatedAt: new Date().toISOString(),
    ...over,
  };
}

describe('clipboardSync (pure)', () => {
  it('bumps epoch', () => {
    expect(nextEpoch(1)).toBe(2);
  });

  it('rejects frames from a prior epoch', () => {
    expect(isClipboardFresh({ epoch: 1, seq: 5 }, 2, 0)).toBe(false); // stale epoch
    expect(isClipboardFresh({ epoch: 2, seq: 1 }, 2, 0)).toBe(true);  // fresh
    expect(isClipboardFresh({ epoch: 2, seq: 1 }, 2, 1)).toBe(false); // not newer
  });

  it('clipboardOperational fails closed', () => {
    expect(clipboardOperational(null, true)).toBe(false);
    expect(clipboardOperational(policy({ trustStatus: 'blocked' }), true)).toBe(false);
    expect(clipboardOperational(policy(), false)).toBe(false);
    expect(clipboardOperational(policy(), true)).toBe(true);
  });
});

describe('ClipboardResyncGuard', () => {
  it('does not send or accept when not operational', () => {
    const g = new ClipboardResyncGuard();
```

```
    g.refreshOperational(null, true); // fail closed
    expect(g.stampOutgoing()).toBeNull();
    expect(g.acceptIncoming({ epoch: g.currentEpoch(), seq: 1 })).toBe(false);
  });

  it('stamps + accepts fresh frames when operational', () => {
    const g = new ClipboardResyncGuard();
    g.refreshOperational(policy(), true);
    const meta = g.stampOutgoing();
    expect(meta).not.toBeNull();
    expect(g.acceptIncoming({ epoch: g.currentEpoch(), seq: (meta!.seq) + 1 })).toBe(true);
  });

  it('rejects stale frames after an epoch bump (reconnect)', () => {
    const g = new ClipboardResyncGuard();
    g.refreshOperational(policy(), true);
    const stale = { epoch: g.currentEpoch(), seq: 99 };
    g.bumpEpoch(); // reconnect
    g.refreshOperational(policy(), true);
    expect(g.acceptIncoming(stale)).toBe(false); // prior-epoch frame rejected
  });
});
```

Part G — Reports

FILE: generated-pack19-desktop-transfer-clipboard-reconnect-merge-summary.md

```
# Pack 19 (Desktop) — File-Transfer Resume + Clipboard Resync · Merge Summary

## What this slice does
Continues Desktop Packs 15 → 18. Makes the two stateful interactive features
reconnect-safe:
1. FileTransferResumeGuard pauses in-flight transfers on a transport drop and,
   after the Pack 18 policy re-check, resumes only those the policy still permits
   (aborts/denies the rest). Fail closed on missing/blocked policy.
2. ClipboardResyncGuard stamps every clipboard frame with an epoch + seq and bumps
   the epoch on reconnect, so a clipboard captured before a policy change can never
   replay afterward. Fail closed when clipboard is not operational.

## Files created (8)
- packages/shared/src/permissions/fileTransferState.ts
- packages/shared/src/permissions/clipboardSync.ts
- apps/desktop/src/renderer/src/services/fileTransferResumeGuard.ts
- apps/desktop/src/renderer/src/services/clipboardResyncGuard.ts
- apps/desktop/src/renderer/src/components/TransferResumePanel.tsx
- tests/permissions/fileTransferState.test.ts
- tests/permissions/fileTransferResumeGuard.test.ts
- tests/permissions/clipboardSync.test.ts

## Files modified (4)
- packages/shared/src/permissions/index.ts — export fileTransferState + clipboardSync.
- apps/desktop/src/renderer/src/services/reconnectPolicyController.ts — pause/resume transfers + bump/refresh clipboard
on drop/restore.
- apps/desktop/src/renderer/src/services/sessionDataChannel.ts — stamp/gate clipboard frames + feed transfer progress to
the guard.
```

```
- apps/desktop/src/renderer/src/components/RemoteSessionView.tsx — transfer resume panel.

## Files intentionally NOT touched
- main/index.ts, preload/index.ts — no native executor / no bridge.
- webrtc.ts, permissionState.ts — Pack 15/16/17 gates unchanged.
- apps/api/**, apps/web/** — purely session-local reconnect behavior.

## Remaining manual wiring risks
1. Construct FileTransferResumeGuard + ClipboardResyncGuard at session start; pass into ReconnectPolicyController deps (+ hostAccepted getter).
2. Feed real transfer progress/complete events into the guard's track()/remove().
3. Add envelope meta (epoch+seq) to clipboard frames; stamp on send, check on receive.
4. Wire resumeTransfer/abortTransfer to your actual file-transfer implementation (byte-offset resume).

## Verify
- pnpm --filter @remotedesk/shared build && pnpm -r test
- Manual: start a transfer, drop the socket -> it pauses; reconnect with file transfer still allowed -> resumes from offset; flip the policy to disallow -> it's denied + aborted. Copy before a drop, change policy, reconnect -> old clipboard frame is rejected as stale.
```

FILE: generated-pack19-desktop-transfer-clipboard-reconnect-code-review.md

```
# Pack 19 (Desktop) — Code Review Notes

## Strengths
- Reuses the SAME shared evaluators (canEnableFileTransfer / canEnableClipboard), so resume/resync can't be looser than the live policy.
- Transfers pause on drop and only resume after the Pack 18 re-check; a policy that flipped while offline denies + aborts.
- Clipboard epoch bump makes stale replay structurally impossible, not just timing-dependent.
- Both guards fail closed: missing policy / blocked / not-accepted => no resume, no clipboard.
- Pure state logic fully unit-tested; guards tested with stubs (no live channel).

## Reviewer checklist
- [ ] Guards constructed at session start + passed into ReconnectPolicyController deps.
- [ ] hostAccepted getter wired so resume decisions see real acceptance.
- [ ] Clipboard frames carry epoch+seq; send stamps, receive checks freshness.
- [ ] resumeTransfer uses the byte offset; abortTransfer cleans up partial files.
- [ ] No file contents / clipboard contents logged (names + counts only).

## Safety
- File transfer never auto-resumes on a revoked/blocked/missing policy.
- Clipboard cannot replay a pre-reconnect payload (epoch invalidation).
- Native OS execution remains disabled/no-op (untouched).
```

FILE: generated-pack19-desktop-transfer-clipboard-reconnect-manifest.json

```
{
  "pack": "pack19-desktop-transfer-clipboard-reconnect",
  "title": "File-Transfer Resumption Guards + Clipboard Resync After Reconnect",
  "actualFileCount": 12,
  "filesCreated": [
    "packages/shared/src/permissions/fileTransferState.ts",
    "packages/shared/src/permissions/clipboardSync.ts",
    "apps/desktop/src/renderer/src/services/fileTransferResumeGuard.ts",
    "apps/desktop/src/renderer/src/services/clipboardResyncGuard.ts",
```

```
      "apps/desktop/src/renderer/src/components/TransferResumePanel.tsx",
      "tests/permissions/fileTransferState.test.ts",
      "tests/permissions/fileTransferResumeGuard.test.ts",
      "tests/permissions/clipboardSync.test.ts"
    ],
    "filesModified": [
      "packages/shared/src/permissions/index.ts",
      "apps/desktop/src/renderer/src/services/reconnectPolicyController.ts",
      "apps/desktop/src/renderer/src/services/sessionDataChannel.ts",
      "apps/desktop/src/renderer/src/components/RemoteSessionView.tsx"
    ],
    "filesNotTouched": [
      "apps/desktop/src/main/index.ts",
      "apps/desktop/src/preload/index.ts",
      "apps/desktop/src/renderer/src/services/webrtc.ts",
      "apps/desktop/src/renderer/src/services/permissionState.ts",
      "apps/api/**",
      "apps/web/**"
    ],
    "dependenciesRequired": {
      "new": [],
      "existing": ["@remotedesk/shared", "react", "vitest"],
      "assumedFromProject": [
        "Desktop Pack 18 ReconnectPolicyController",
        "Desktop Pack 16 channelFeatureGate",
        "existing file-transfer impl with progress/complete events + byte-offset resume",
        "existing clipboard data channel"
      ]
    },
    "knownLimitations": [
      "Byte-offset resume depends on the existing transfer implementation supporting it; otherwise transfers restart.",
      "Clipboard epoch is session-local in memory; not persisted across full app restarts.",
      "Guards are reconnect-safety only; they do not add new transfer/clipboard transport.",
      "Native OS input execution remains disabled/no-op."
    ],
    "remainingManualWiringRisks": [
      "Construct + wire both guards into the reconnect controller with a hostAccepted getter.",
      "Add epoch+seq envelope meta to clipboard frames (stamp on send, check on receive).",
      "Point resumeTransfer/abortTransfer at the real transfer implementation."
    ],
    "estimatedCompletionAfterCarefulMerge": "68-74%"
  }
```

Desktop Pack 19 done: 12 files (8 created, 4 modified). The two stateful interactive features are now reconnect-safe. In-flight file transfers pause the instant the transport drops and only resume after Pack 18's policy re-check, and only for transfers the policy still permits, the rest are denied and aborted. Clipboard frames carry an epoch + sequence; reconnect bumps the epoch so a clipboard captured before a policy change can never replay afterward.

The clean bit: both guards reuse the exact same shared evaluators (`canEnableFileTransfer`, `canEnableClipboard`), so resume/resync can never be looser than the live policy, and the clipboard epoch bump makes stale replay structurally impossible rather than just timing-dependent. Everything fails closed on a missing/blocked/not-accepted policy, and native OS execution stays untouched.

Next slice = viewer-side trust prompt + first-connect device fingerprint, or unattended-access pre-authorization flow. Bolo.